Kiến trúc đọc dòng động (Dynamic Line Buffer) và So khớp nhị phân dòng lệnh — (diff.c)

1. Tiếp nhận Luồng File

Kiểm tra hợp lệ & Đọc byte stream

File 1 (fd1 = open(f1))

Kiểm tra: fstat(fd1) == T_FILE

`"apple\nbanana\ncarrot\n"`

File 2 (fd2 = open(f2))

Kiểm tra: fstat(fd2) == T_FILE

`"apple\nblueberry\ncarrot\ndate"`

Rào cản an toàn xv6:

- Từ chối thư mục / device (st.type)
- Đọc tuần tự từng byte tìm '\n'
- Dừng khi gặp EOF (read() == 0)

1B

2. Cơ chế readline() Tự co giãn

Chống tràn Buffer với bộ nhớ động

Khởi tạo ban đầu (Lazy Alloc):

```
cap = 128; buf = malloc(cap);
```

Tối ưu bộ nhớ heap cho hầu hết các dòng

Tự động tăng trưởng (Reallocation):

```
new_cap = cap * 2; malloc(new_cap);
memmove(new_buf, buf, len); free(buf);
```

✓ Tự co giãn theo độ dài dòng thực tế

Giám sát cờ trạng thái dòng:

- has_newline = 1 nếu ký tự cuối là '\n'
- has_newline = 0 nếu chạm EOF trước khi '\n'

3. Bộ so khớp dòng & Xử lý ca biên

In định dạng chuẩn hóa, phát hiện EOF sớm & Quiet Mode

Trường hợp 1: Nội dung dòng lệch nhau (Dòng 2)

```
f1:2: < banana\n
f2:2: > blueberry\n
```

Trường hợp 2: Lệch số dòng / File kết thúc sớm (Dòng 4)

```
f1:4: < EOF
f2:4: > date (diff: f2:4: no newline at end of file)
```

Tối ưu hóa ngắt sớm (-q) & Exit Code chuẩn Unix:

- Cờ -q (Quiet): Ngay khi phát hiện khác biệt đầu tiên, in "diff: files differ" và dừng vòng lặp lập tức.
- Thu hồi tài nguyên: free(buf1); free(buf2); close(fd1); close(fd2);

```
exit(0): Trùng khớp | exit(1): Khác biệt | exit(2): Lỗi I/O
```